Vectores y matrices en C++

Resumen en mis palabras. El código completo está en vectores_y_matrices.cpp

1. Vectores

Un vector es una fila de casillas numeradas **desde 0**. En C++ uso `std::vector` porque crece y sabe su tamaño (el arreglo clásico `int a[5]` es fijo y no avisa si me salgo del rango).

```
#include <vector>
vector<int> v = {3, 4, 5};
v.push_back(2);     // agrega al final -> {3, 4, 5, 2}
v.size();           // 4
v[1];               // 4  (indices desde 0)
```

Para recorrerlo uso un *for*. Simulación sumando {3, 4, 5}:

```
int suma = 0;
for (size_t i = 0; i < v.size(); i++) suma += v[i];
```

i	v[i]	suma
0	3	3
1	4	7
2	5	12

Al pasarlo a una función uso `const vector<int>&`: el & evita copiar todo el vector y el `const` promete no modificarlo.

2. Vectores de álgebra lineal

Suma: `r[i] = a[i] + b[i]`. Producto punto: multiplico posición por posición y sumo, y el resultado es **un número**. Con u = (1,2,3) y w = (4,5,6): 1*4 + 2*5 + 3*6 = **32**.

```
double productoPunto(const Vector& a, const Vector& b) {
    double suma = 0;
    for (size_t i = 0; i < a.size(); i++) suma += a[i] * b[i];
    return suma;
}
```

3. Matrices

Una matriz es un **vector de vectores** (cada elemento es una fila). Accedo con `A[fila][columna]`; `A.size()` son las filas y `A[0].size()` las columnas.

```
Matriz A = {{1, 2, 3},
            {4, 5, 6}};     // 2x3,  A[0][2] vale 3
```

Para recorrerla uso dos ciclos: el de afuera son las filas (i) y el de adentro las columnas (j). El de adentro da todas sus vueltas por cada vuelta del de afuera.

4. Operaciones con matrices

Suma: `C[i][j] = A[i][j] + B[i][j]` (misma dimensión). **Transpuesta:** `T[j][i] = A[i][j]` (la 2x3 pasa a 3x2, así que creo la matriz resultado con dimensiones invertidas).

Producto: las columnas de A deben ser iguales a las filas de B. Cada casilla es la fila i de A por la columna j de B, por eso lleva tres ciclos:

```
for (int i = 0; i < filas; i++)
    for (int j = 0; j < cols; j++)
        for (int k = 0; k < n; k++)
            C[i][j] += A[i][k] * B[k][j];
```

Simulación de A (2x3) por A^T (3x2), que da una 2x2:

Casilla	Cuenta	Resultado
C[0][0]	1*1 + 2*2 + 3*3	14
C[0][1]	1*4 + 2*5 + 3*6	32
C[1][0]	4*1 + 5*2 + 6*3	32
C[1][1]	4*4 + 5*5 + 6*6	77

Ojo: A*B no es lo mismo que B*A, y a veces uno de los dos ni se puede calcular por las dimensiones.

5. Errores típicos

Error	Solución
Contar desde 1	Los índices van de 0 a size()-1
Usar i <= v.size()	Usar siempre i < v.size()
Confundir filas con columnas	A.size() = filas, A[0].size() = columnas
No inicializar el acumulador	double suma = 0; antes del ciclo
Pasar el vector sin &	Usar const vector<T>&

Conclusión

Un vector es una lista y una matriz es una lista de listas, así que casi todo se resuelve con un ciclo (vectores) o con dos o tres ciclos anidados (matrices). Lo que más me ayudó fue simular a mano cada vuelta antes de correr el programa. Lo que más hay que cuidar es el rango de los índices y que las dimensiones coincidan antes de sumar o multiplicar.